

Python seaborn Library

Seaborn library aims to make a more attractive visualization of the central part of understanding and exploring data. It is built on the core of the matplotlib library and also provides dataset-oriented APIs.

Seaborn is also closely integrated with the Panda's data structures, and with this, we can easily jump between the various different visual representations for a given variable to better understand the provided dataset.

Categories of Plots in Python's seaborn library

Plots are generally used to make visualization of the relationships between the given variables. These variables can either be a category like a group, division, or class or can be completely numerical variables. There are various different categories of plots that we can create using the seaborn library.

In the seaborn library, the plot that we create is divided into the following various categories:

Distribution plots: This type of plot is used for examining both types of distributions, i.e., univariate and bivariate distribution.

Relational plots: This type of plot is used to understand the relation between the two given variables.

Regression plots: Regression plots in the seaborn library are primarily intended to add an additional visual guide that will help to emphasize dataset patterns during the analysis of exploratory data.

Categorical plots: The categorical plots are used to deals with categories of variables and how we can visualize them.

Multi-plot grids: The multi-plot grids are also a type of plot that is a useful approach is to draw multiple instances for the same plot with different subsets of a single dataset.

Matrix plots: The matrix plots are a type of arrays of the scatterplots.

Plotting Chart Using seaborn Library

1. Line plot:

The flights dataset has 10 years of monthly airline passenger data:

```
In [2]: import seaborn as sns
# Importing matplotlib library to show graph in output
import matplotlib.pyplot as plt
# Setting style with set() function
sns.set(style="dark")
# Using dataset() function to declare data type
flights = sns.load_dataset("flights")
flights.head()
```

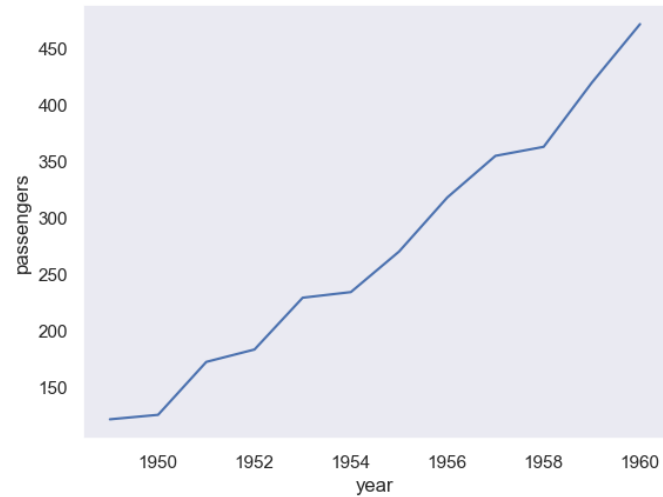
```
Out[2]:
```

	year	month	passengers
0	1949	Jan	112
1	1949	Feb	118
2	1949	Mar	132
3	1949	Apr	129
4	1949	May	121

To draw a line plot using long-form data, assign the x and y variables:

```
In [3]: may_flights = flights.query("month == 'May'")
sns.lineplot(data=may_flights, x="year", y="passengers")
```

```
Out[3]: <Axes: xlabel='year', ylabel='passengers'>
```



Pivot the dataframe to a wide-form representation:

```
In [4]: flights_wide = flights.pivot(index="year", columns="month", values="passengers")
flights_wide.head()
```

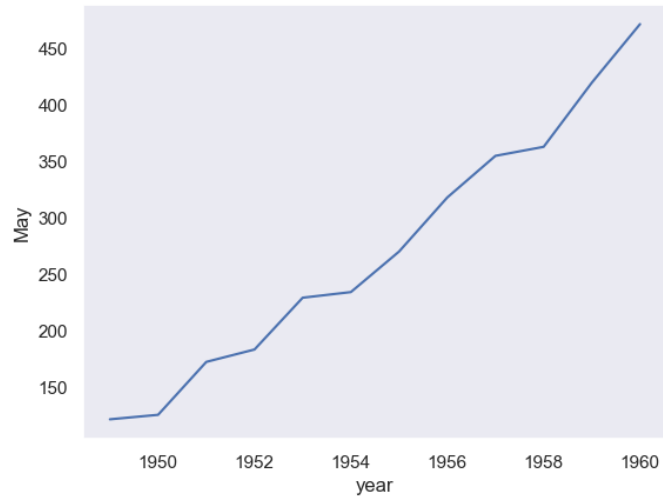
```
Out[4]:
```

	month	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
year													
1949		112	118	132	129	121	135	148	148	136	119	104	118
1950		115	126	141	135	125	149	170	170	158	133	114	140
1951		145	150	178	163	172	178	199	199	184	162	146	166
1952		171	180	193	181	183	218	230	242	209	191	172	194
1953		196	196	236	235	229	243	264	272	237	211	180	201

To plot a single vector, pass it to data. If the vector is a pandas.Series, it will be plotted against its index:

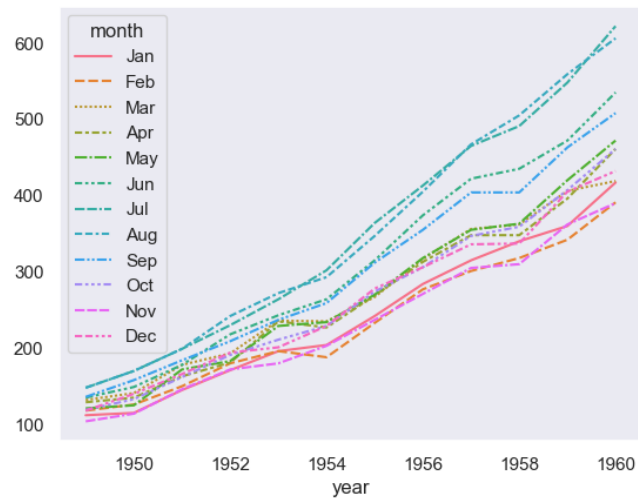
```
In [5]: sns.lineplot(data=flights_wide["May"])
```

```
Out[5]: <Axes: xlabel='year', ylabel='May'>
```



```
In [6]: sns.lineplot(data=flights_wide)
```

```
Out[6]: <Axes: xlabel='year'>
```



2. Dist plot:

These kind of plots are used to visualize the distribution of the features. Understanding how your data is distributed can help you to determine the range of values, their central tendency, or if they maybe inclined in one direction and also spotting outliers.

- Distribution plots functions:
- `displot()`
- `jointplot()`
- `pairplot()`

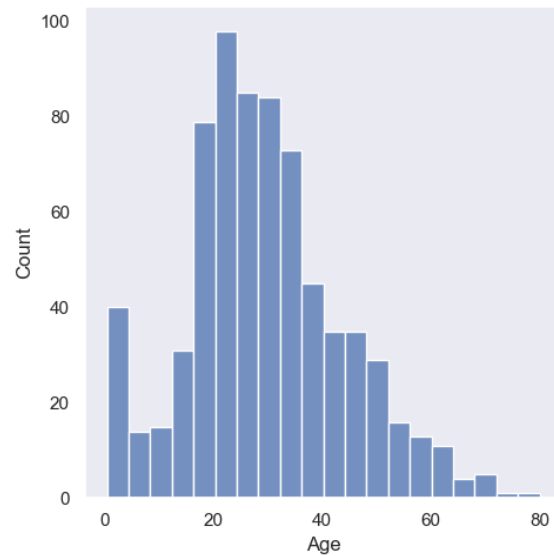
- rugplot()
- kdeplot

Plotting Histograms with `displot()` and `histplot()` Histogram is a bar plot whose x-axis is a variable or a feature and y-axis being the count of values of that particular variable. By default, `displot()` plots the histogram.

Histogram is a bar plot whose x-axis is a variable or a feature and y-axis being the count of values of that particular variable. By default, `displot()` plots the histogram.

```
In [12]: import pandas as pd
titanic=pd.read_csv("Titanic.csv")
titanic.head()
sns.displot(titanic['Age'])
```

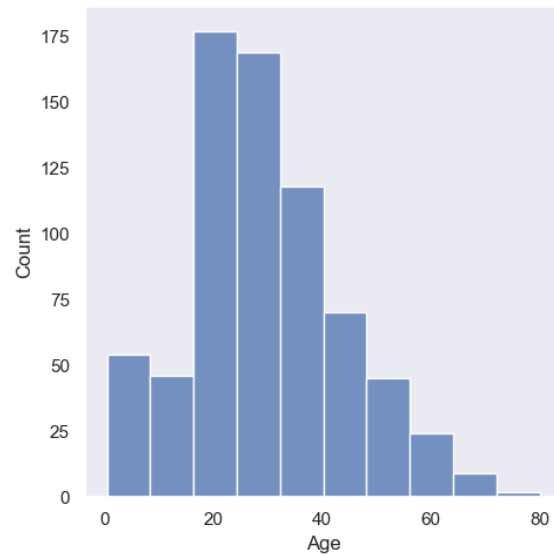
```
Out[12]: <seaborn.axisgrid.FacetGrid at 0xd4d25d0850>
```



By default, Kernel Density Estimator(KDE) is True. By setting it off, we should remain with the histograms. A histogram displays data by grouping data into bins. We can set the bins to a value of choice.

```
In [14]: sns.displot(titanic['Age'], kde=False, bins=10)
```

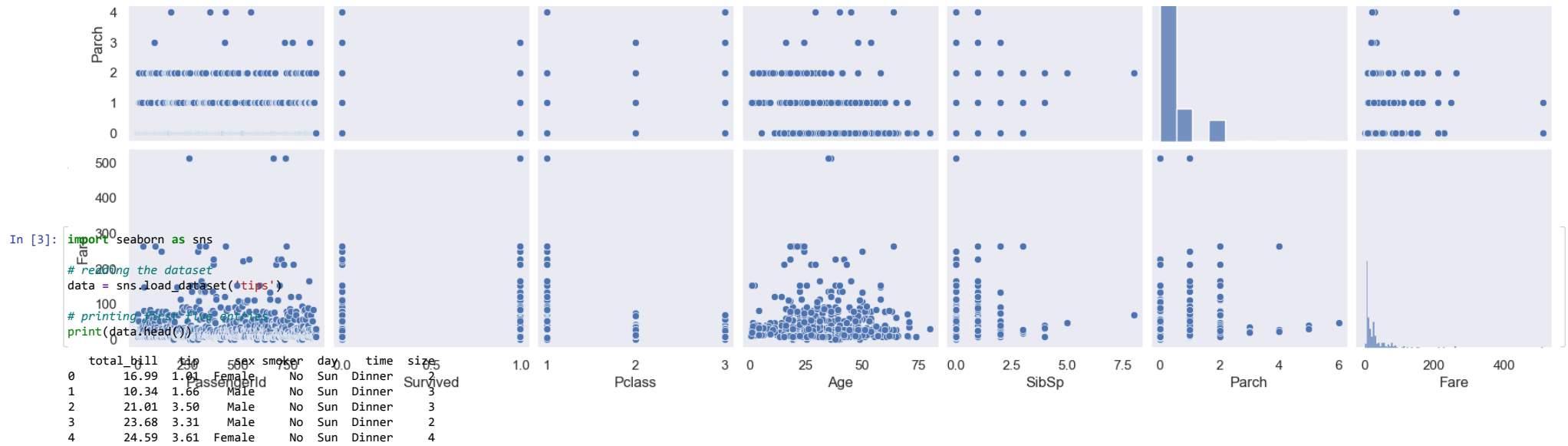
```
Out[14]: <seaborn.axisgrid.FacetGrid at 0xd4d3270d50>
```



In [15]: `sns.pairplot(titanic)`

Out[15]: `<seaborn.axisgrid.PairGrid at 0xd4d34a71d0>`





To draw the relational plots seaborn provides three functions. These are:

- relplot()
- scatterplot()
- lineplot()

Seaborn.relplot()

This function provides us the access to some other different axes-level functions which shows the relationships between two variables with semantic mappings of subsets.

seaborn.relplot(x=None, y=None, data=None, **kwargs)

Example 1: Visualizing the most basic plot to show all the data points in tips dataset.

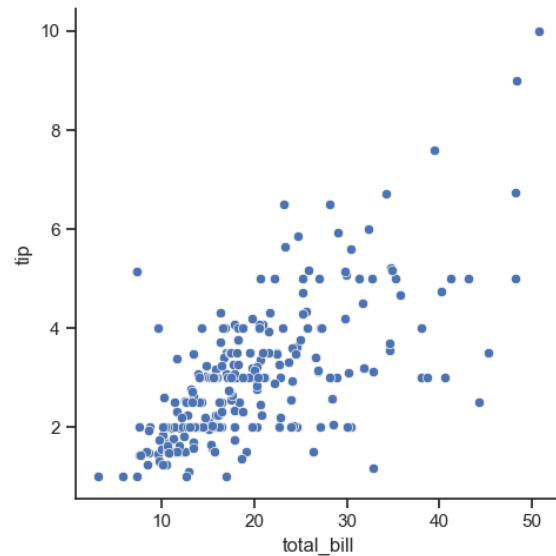
```
In [4]: # importing the library
import seaborn as sns

# selecting style
sns.set(style="ticks")

# reading the dataset
tips = sns.load_dataset('tips')

# plotting a simple visualization of data points
sns.relplot(x="total_bill", y="tip", data=tips)
```

Out[4]: <seaborn.axisgrid.FacetGrid at 0x77bc6b79d0>



Visualizing categorical data

Categorical scatterplots:

stripplot() (with kind="strip"; the default)

swarmplot() (with kind="swarm")

Categorical distribution plots:

boxplot() (with kind="box")

violinplot() (with kind="violin")

boxenplot() (with kind="boxen")

Categorical estimate plots:

```
pointplot() (with kind="point")
```

```
barplot() (with kind="bar")
```

```
countplot() (with kind="count")
```

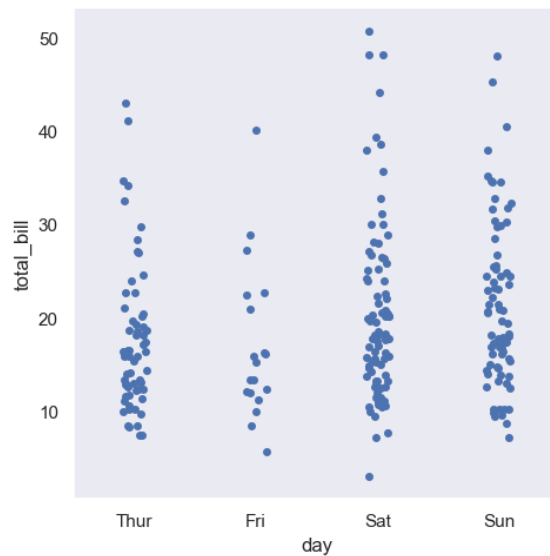
Categorical scatterplots

The default representation of the data in `catplot()` uses a scatterplot. There are actually two different categorical scatter plots in seaborn. They take different approaches to resolving the main challenge in representing categorical data with a scatter plot, which is that all of the points belonging to one category would fall on the same position along the axis corresponding to the categorical variable. The approach used by `stripplot()`, which is the default "kind" in `catplot()` is to adjust the positions of points on the categorical axis with a small amount of random "jitter":

```
In [16]: tips = sns.load_dataset("tips")
print(tips.columns)
sns.catplot(data=tips, x="day", y="total_bill")
```

```
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'], dtype='object')
```

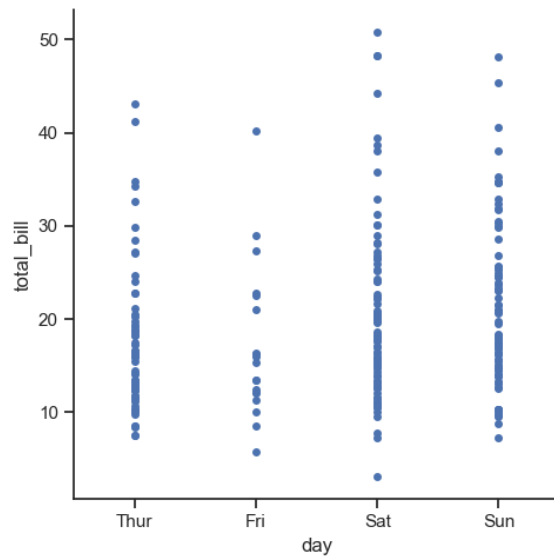
```
Out[16]: <seaborn.axisgrid.FacetGrid at 0xd4d5232810>
```



The jitter parameter controls the magnitude of jitter or disables it altogether:

```
In [6]: sns.catplot(data=tips, x="day", y="total_bill", jitter=False)
```

```
Out[6]: <seaborn.axisgrid.FacetGrid at 0x77bcc53e50>
```



Comparing distributions

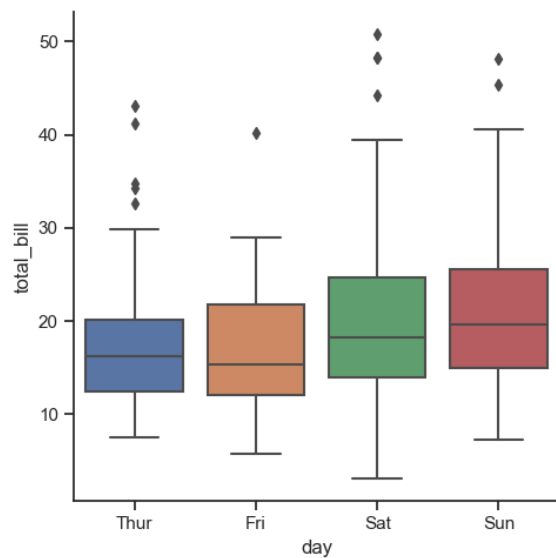
As the size of the dataset grows, categorical scatter plots become limited in the information they can provide about the distribution of values within each category. When this happens, there are several approaches for summarizing the distributional information in ways that facilitate easy comparisons across the category levels.

Boxplots

The first is the familiar `boxplot()`. This kind of plot shows the three quartile values of the distribution along with extreme values. The "whiskers" extend to points that lie within 1.5 IQRs of the lower and upper quartile, and then observations that fall outside this range are displayed independently. This means that each value in the boxplot corresponds to an actual observation in the data.

```
In [8]: sns.catplot(data=tips, x="day", y="total_bill", kind="box")
```

```
Out[8]: <seaborn.axisgrid.FacetGrid at 0x77bd49f0d0>
```



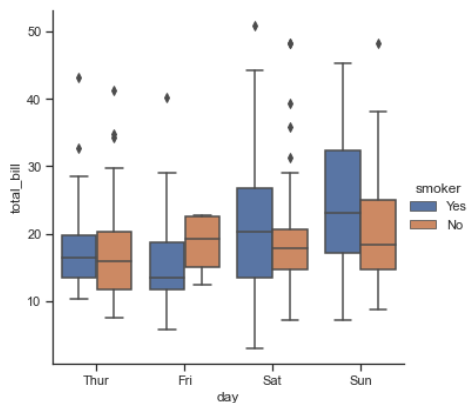
When adding a hue semantic, the box for each level of the semantic variable is moved along the categorical axis so they don't overlap:

Functionality of hue="smoker":

Color Encoding: Seaborn will assign different colors to distinguish between "smoker" and "non-smoker" groups (assuming "smoker" is a binary categorical variable like in the common "tips" dataset).

```
In [9]: sns.catplot(data=tips, x="day", y="total_bill", hue="smoker", kind="box")
```

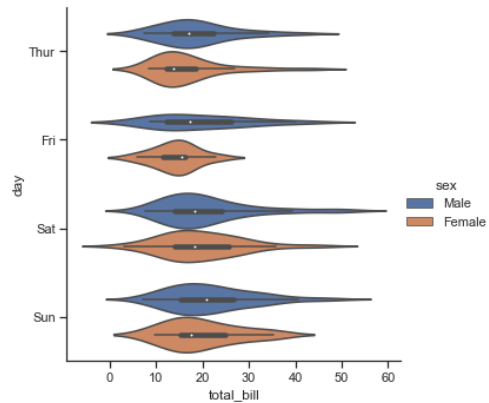
```
Out[9]: <seaborn.axisgrid.FacetGrid at 0xb4c8819fa0>
```



A violin plot is a graphical representation of the distribution of numerical data, similar to a box plot but with the added benefit of showing the probability density of the data. It combines a kernel density plot with a box plot, revealing peaks and trends in the data distribution that a box plot alone might miss. Violin plots are particularly useful for comparing distributions across multiple groups or categories, making them a powerful tool for exploratory data analysis. ## Violinplots

```
In [10]: sns.catplot(  
    data=tips, x="total_bill", y="day", hue="sex", kind="violin",  
    )
```

```
Out[10]: <seaborn.axisgrid.FacetGrid at 0xb4c87ac130>
```



Estimating central tendency

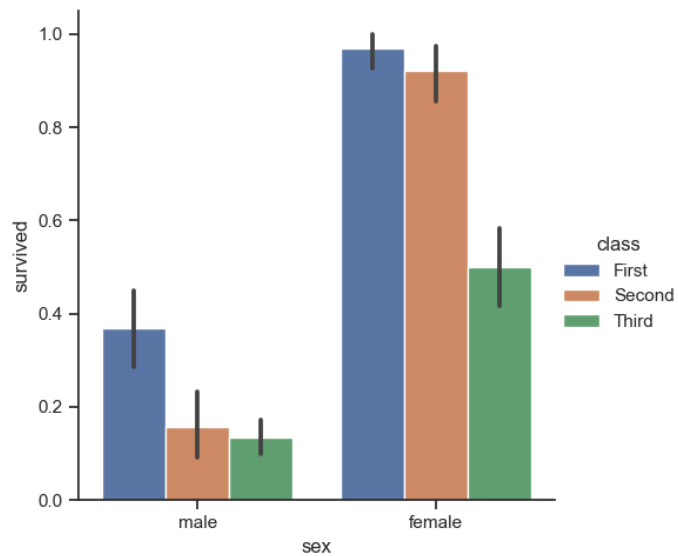
For other applications, rather than showing the distribution within each category, you might want to show an estimate of the central tendency of the values. Seaborn has two main ways to show this information. Importantly, the basic API for these functions is identical to that for the ones discussed above.

Bar plots

A familiar style of plot that accomplishes this goal is a bar plot. In seaborn, the `barplot()` function operates on a full dataset and applies a function to obtain the estimate (taking the mean by default). When there are multiple observations in each category, it also uses bootstrapping to compute a confidence interval around the estimate, which is plotted using error bars:

```
In [10]: titanic = sns.load_dataset("titanic")
sns.catplot(data=titanic, x="sex", y="survived", hue="class", kind="bar")
```

```
Out[10]: <seaborn.axisgrid.FacetGrid at 0x77bd95a390>
```

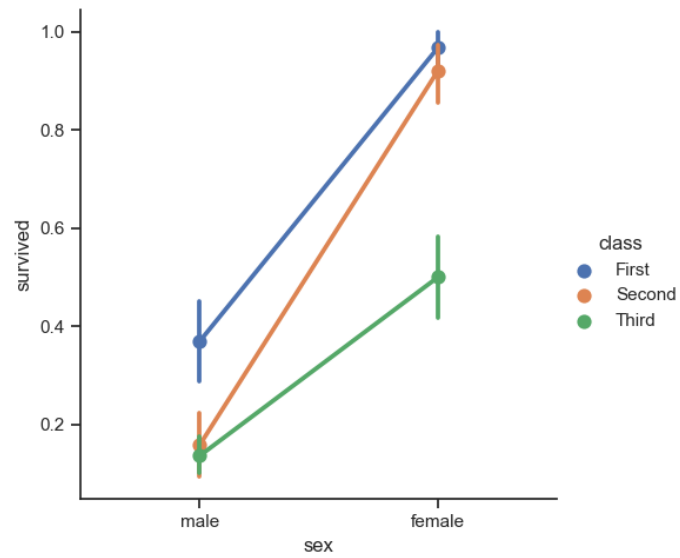


Point plots

An alternative style for visualizing the same information is offered by the `pointplot()` function. This function also encodes the value of the estimate with height on the other axis, but rather than showing a full bar, it plots the point estimate and confidence interval. Additionally, `pointplot()` connects points from the same hue category. This makes it easy to see how the main relationship is changing as a function of the hue semantic, because your eyes are quite good at picking up on differences of slopes:

```
In [11]: sns.catplot(data=titanic, x="sex", y="survived", hue="class", kind="point")
```

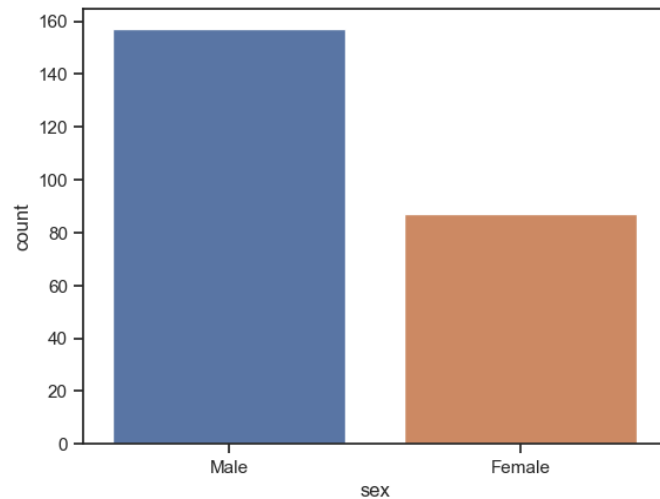
```
Out[11]: <seaborn.axisgrid.FacetGrid at 0x77bd509290>
```



While the categorical functions lack the style semantic of the relational functions, it can still be a good idea to vary the marker and/or linestyle along with the hue to make figures that are maximally accessible and reproduce well in black and white:

```
In [12]: import seaborn as sns  
  
# import done to avoid warnings  
from warnings import filterwarnings  
  
# reading the dataset  
df = sns.load_dataset('tips')  
  
# first five entries if the dataset  
df.head()  
sns.countplot(x='sex', data=df)
```

Out[12]: <Axes: xlabel='sex', ylabel='count'>



Plotting pairwise data relationships

PairGrid also allows you to quickly draw a grid of small subplots using the same plot type to visualize data in each. In a PairGrid, each row and column is assigned to a different variable, so the resulting plot shows each pairwise relationship in the dataset. This style of plot is sometimes called a "scatterplot matrix", as this is the most common way to show each relationship, but PairGrid is not limited to scatterplots.

```
In [13]: iris = sns.load_dataset("iris")  
g = sns.PairGrid(iris)  
g.map(sns.scatterplot)
```

```
Out[13]: <seaborn.axisgrid.PairGrid at 0x77bd589190>
```

